



Lenguaje de programación de alto nivel, similar a MATLAB®, enfocado en cálculos numéricos y científicos.

Tu herramienta libre para el cálculo numérico y el análisis de datos.

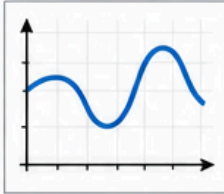
¿QUÉ ES GNU OCTAVE?

GNU Octave es un lenguaje de programación de alto nivel, principalmente orientado a **cálculos numéricos**.

Su sintaxis es muy similar a MATLAB®, por lo que es una excelente alternativa **libre y de código abierto**.

¿PARA QUÉ SIRVE?

- ✓ Cálculos matemáticos
- ✓ Álgebra lineal
- ✓ Análisis de datos
- ✓ Visualización de gráficas
- ✓ Desarrollo de algoritmos
- ✓ Simulación y control



VENTAJAS

- Gratuito y de código abierto
- Disponible en Windows, Linux y macOS
- Compatible con muchas funciones de MATLAB®
- Comunidad activa y en constante desarrollo
- Ideal para enseñanza e investigación



Potente, flexible y 100% libre.



ENTORNO DE TRABAJO

Octave cuenta con una interfaz gráfica intuitiva que facilita el trabajo.

File Browser

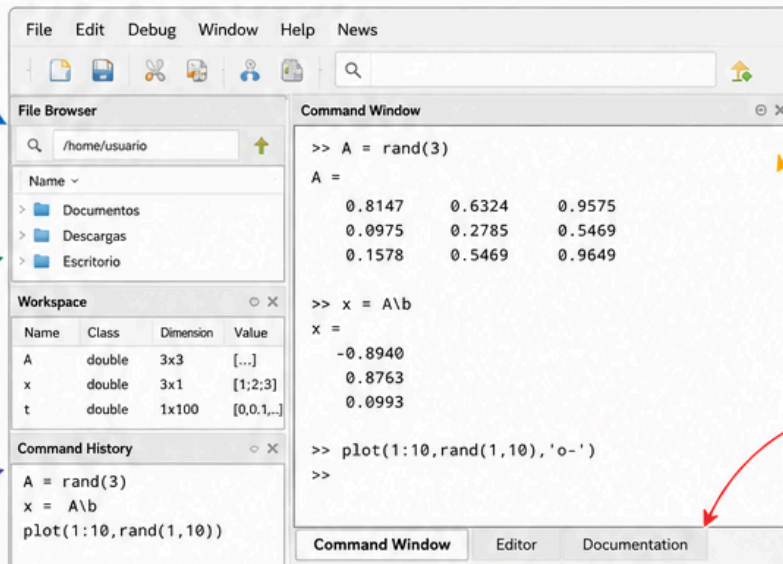
Explora y administra archivos y carpetas.

Workspace

Muestra las variables actuales en memoria.

Command History

Historial de comandos ejecutados.



Command Window

Escriba comandos y vea los resultados aquí.

Editor / Documentation

Edite scripts y funciones, y consulte la ayuda integrada.

PRIMEROS COMANDOS

Comando	Descripción	
<code>clc</code>	Limpia la ventana de comandos.	
<code>clear</code>	Elimina todas las variables del Workspace.	
<code>close all</code>	Cierra todas las ventanas de figuras.	
<code>who</code>	Muestra las variables en memoria.	
<code>whos</code>	Muestra información detallada de las variables.	
<code>help nombre</code>	Muestra ayuda sobre un comando o función.	
<code>pkg list</code>	Lista los paquetes instalados.	



EJEMPLO RÁPIDO

```
1 clc; clear; close all; % limpiar
2 A = rand(3) % matriz aleatoria 3x3
3 b = [1; 2; 3]; % vector columna
4 x = A\b % solución del sistema Ax=b
5 disp('Solución:');
6 disp(x)
7 plot(1:10, rand(1,10), 'o-'); % gráfica
8 title('Ejemplo en Octave')
9 xlabel('Índice')
10 ylabel('Valor')
11 grid on
```

¡Escriba, experimente y descubra!



CONSEJOS

- Terminar cada comando con punto y coma (;) para evitar imprimir resultados innecesarios.
- Use comentarios (%) para explicar su código.
- Octave distingue entre mayúsculas y minúsculas.



NOTA IMPORTANTE

Octave está en constante desarrollo. Algunos paquetes adicionales se instalan con: `pkg install -forge nombrePaquete`



MÁS INFORMACIÓN

Sitio oficial de GNU Octave:
<https://octave.org>





1. VECTORES

Un vector es un arreglo unidimensional de números.

CREACIÓN DE VECTORES

```
v = [0.1 0.2 0.3 0.4 0.5]
           % vector fila

w = [0.1; 0.2; 0.3; 0.4; 0.5]
           % vector columna

v = 0.1:0.1:0.5
           % 0.1 0.2 0.3 0.4 0.5

v = linspace(0,1,6)
           % 0 0.2 0.4 0.6 0.8 1

v = logspace(-1,1,3)
           % 10^-1 10^0 10^1
           % 0.1 1 10
```

CONCATENACIÓN

```
u = [0.6 0.7 0.8]
v = [0.1 0.2 0.3 0.4 0.5]
v u
% horizontal → [0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8]

v; u
% vertical → columna 8×1
```

VECTOR VACÍO

```
v = []
           % vector sin elementos
```

OPERACIONES BÁSICAS

```
v = [0.1 0.2 0.3 0.4 0.5]

length(v) % longitud
ans = 5

v(3) % elemento 3
ans = 0.3

v([1 4 5]) % seleccionar
ans = 0.1 0.4 0.5

v(2:4) % por rango
ans = 0.2 0.3 0.4

v' % transpuesta (fila→columna)
ans =
    0.1
    0.2
    0.3
    0.4
    0.5

v(end) % último elemento
ans = 0.5

sum(v) % suma de elementos
ans = 1.5

mean(v) % promedio
ans = 0.3

max(v), min(v) % máximo y mínimo
ans = 0.5, 0.1
```



2. MATRICES

Una matriz es un arreglo bidimensional de números.

CREACIÓN DE MATRICES

```
A = [1 2 3; 4 5 6; 7 8 9]

A =
    1    2    3
    4    5    6
    7    8    9

B = zeros(3,4)
% matriz 3×4 de ceros
B =
    0    0    0    0
    0    0    0    0
    0    0    0    0

C = ones(2,3)
% matriz 2×3 de unos
C =
    1    1    1
    1    1    1

D = eye(3)
% matriz identidad 3×3
D =
    1    0    0
    0    1    0
    0    0    1

E = rand(2,3) % matriz aleatoria
E =
    0.8147    0.6324    0.9575
    0.0975    0.2785    0.5469

F = randi(1:10,2,3) % enteros 1 a 10
F =
    3    8    2
    9    1    6
```

PROPIEDADES IMPORTANTES

```
size(A) % tamaño (filas, columnas)
ans = 3    3

length(A) % mayor dimensión
ans = 3

numel(A) % total de elementos
ans = 9

A(m,n) % elemento en fila m,
columna n

A(:,2) % segunda columna

A(2,:) % segunda fila
```

TRANSUESTA Y CONJUGADA

```
A = [1 2 3; 4 5 6]

A' % transpuesta (conjugada)
A =
    1    4
    2    5
    3    6

A.' % transpuesta NO conjugada
A =
    1    4
    2    5
    3    6
```



TIP: En Octave, los índices comienzan en 1.



TIP: Una matriz de tamaño m×n tiene m filas y n columnas.



3. OPERACIONES CON MATRICES

SUMA Y RESTA

```
A = [1 2 3; 4 6]
B = [6 5; 4 2 1]

A + B
ans =
    7    7    7
    7    7    7

A - B
ans =
   -5   -3   -1
    1    3    5
```

MULTIPLICACIÓN MATRICIAL
(PRODUCTO)

```
A = [1 2 3; 4 6] % 2×3
B = [7 8; 9 11 12] % 3×2

C = A * B % 2×2
C =
    58    64
   139   154
```

OPERACIONES ELEMENTO A ELEMENTO
(mismo tamaño)

```
A = [1 2 3; 4 6]
B = [6 5; 4 2 1]

A .* B % producto elemento a elemento
ans =
    6    10    12
   12    10    6

A ./ B % división elemento a elemento
ans =
    0.1667    0.4000    0.7500
    1.3333    2.5000    6.0000

A .^ 2 % potencia elemento a elemento
ans =
    1    4    9
   16   25   36
```

MULTIPLICACIÓN POR ESCALAR

```
A = [1 2 3; 4 6]

2*A
ans =
    2    4    6
    8   10   12

A/2
ans =
    0.5000    1.0000    1.5000
    2.0000    2.5000    3.0000
```

PRODUCTO PUNTO (DOT)

```
x = [1 2 3]
y = [4 5 6]

x * y' % producto punto
ans = 32

dot(x,y) % equivalente
ans = 32
```



4. FUNCIONES ÚTILES

Función	Descripción
diag(v)	Matriz diagonal con v en la diagonal principal.
diag(A)	Extrae la diagonal principal de A.
reshape(A,m,n)	Cambia el tamaño de A a m×n.
linspace(a,b,n)	Vector con n puntos entre a y b.
sort(v)	Ordena los elementos de v.
flip(v)	Invierte el orden de los elementos.
mean(A)	Promedio de los elementos.
max(A), min(A)	Máximo y mínimo.



5. EJEMPLO RÁPIDO

```
% Vectores
v = [0.1 0.2 0.3 0.4 0.5];
w = linspace(0,1,6);
s = sum(v) % 1.5
m = mean(v) % 0.3

% Matrices
A = [1 2 3; 4 5 6; 7 8 9];
B = ones(3,3);
C = A + B; % suma
D = A*B; % producto matriz
E = A ./ 2; % por escalar
F = A(:,2); % segunda columna (2 5 8)'
```

% SALIDA (ejemplo)

```
s =
    1.5000

m =
    0.3000

F =
    2
    5
    8
```



RECUERDA

- Los índices empiezan en 1.
- Verifica las dimensiones antes de operar matrices.
- Usa .*, ./, .^ para operaciones elemento a elemento.
- A\b es mejor que inv(A)*b para resolver sistemas.



ERRORES FRECUENTES

- Dimensiones incompatibles en operaciones matriciales.
- Usar * cuando se requiere .* (o viceversa).
- Olvidar el punto en la transpuesta: A' (conjugada) vs A.'
- Sobrescribir variables sin querer.



BUENAS PRÁCTICAS

- Nombrar variables de forma significativa.
- Comentar tu código con %.
- Verificar los resultados con tamaño y contenido.
- Guardar y reutilizar funciones y scripts.

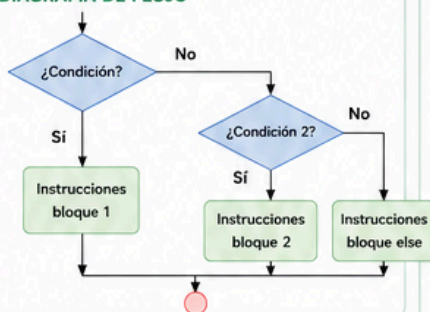


**1 CONDICIONALES: if, elseif, else**

Permiten ejecutar bloques de código según se cumpla una condición.

SINTAXIS

```
if condición
    instrucciones
elseif condición2
    instrucciones
else
    instrucciones
endif
```

DIAGRAMA DE FLUJO**EJEMPLO**

```
x = 7;
if x > 0
    disp('x es positivo'); % condición verdadera
elseif x < 0
    disp('x es negativo'); % condición falsa
else
    disp('x es cero'); % condición falsa
endif
```

RECUERDA: En Octave, es obligatorio cerrar la estructura con **endif**.**2 SELECCIÓN MÚLTIPLE: switch, case, otherwise**

Útil cuando existen varias opciones posibles.

SINTAXIS

```
switch variable
    case valor1
        instrucciones
    case valor2
        instrucciones
    ...
    otherwise
        instrucciones
endswitch
```

EJEMPLO

```
dia = 3;
switch dia
    case 1
        disp('Lunes');
    case 2
        disp('Martes');
    case 3
        disp('Miércoles');
    case 4
        disp('Jueves');
    case 5
        disp('Viernes');
    otherwise
        disp('Fin de semana');
endswitch
```

otherwise se ejecuta si no coincide con ningún **case**.**3 CICLO FOR**

Se usa cuando se conoce el número de iteraciones.

SINTAXIS

```
for variable = inicio:paso:fin
    instrucciones
endfor
```

EJEMPLOS**1 Incremento positivo (por defecto = 1)**

```
for k = 1:5
    disp(k)
endfor % 1 2 3 4 5
```

2 Incremento negativo

```
for k = 5:-1:1
    disp(k)
endfor % 5 4 3 2 1
```

TIP: No modificar la variable de control dentro del ciclo for.**4 CICLO WHILE**

Se ejecuta mientras la condición sea verdadera.

SINTAXIS

```
while condición
    instrucciones
endwhile
```

EJEMPLO: Factorial de n

```
n = 5;
fact = 1;
k = 1;
while k <= n
    fact = fact * k;
    k = k + 1;
endwhile
disp(fact) % 120
```

Calcula $n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$ mientras k sea menor o igual que n. **TIP:** Asegúrate de que la condición cambie para evitar ciclos infinitos.**5 CICLO DO...UNTIL (solo en Octave)**

Se ejecuta al menos una vez y se repite hasta que la condición sea verdadera.

SINTAXIS

```
do
    instrucciones
until (condición)
```

EJEMPLO

```
x = 1;
do
    x = x * 2;
    disp(x)
until (x >= 16) % 2 4 8 16
```

Solo disponible en GNU Octave (no existe en MATLAB).

6 BREAK

Sale inmediatamente del ciclo.

```
for k = 1:100
    if k == 10
        break; % sale cuando k = 10
    end
    disp(k)
endfor
% Imprime 1 2 3 ... 9
```

7 CONTINUE

Salta a la siguiente iteración del ciclo.

```
for k = 1:10
    if mod(k,2) == 0
        continue; % salta números pares
    end
    disp(k) % solo impares
endfor
% Imprime 1 3 5 7 9
```

8 ERRORES FRECUENTES

Usar **=** en lugar de **==**

```
x = 5;
if x = 5 % ¡Incorrecto!
    disp('Igual');
endif
```

Olvidar cerrar la estructura

```
for k = 1:5
    disp(k)
% Falta endfor
```

Debe ser:

```
if x == 5 % Correcto
    disp('Igual');
endif
```

Correcto:

```
for k = 1:5
    disp(k)
endfor
```

 En switch, es buena práctica usar **endswitch** (aunque **end** también funciona).**9 PALABRAS CLAVE ÚTILES**

if, elseif, else	Condicionales.
switch, case, otherwise	Selección múltiple.
for	Repetición por número conocido de iteraciones.
while	Repetición mientras la condición sea verdadera.
do...until	Repetición hasta que la condición sea verdadera (solo Octave).
break	Salte del ciclo inmediatamente.
continue	Salta a la siguiente iteración.

10 BUENAS PRÁCTICAS**1. Usa estructuras simples**Usa **if simple** cuando sea posible. Evita anidar muchos niveles de condicionales o ciclos.**2. Prefiere switch cuando haya muchas opciones**

Mejora la legibilidad y es más fácil de mantener.

**3. Asegura que la condición cambie en while y do...until**

De lo contrario, puedes entrar en un ciclo infinito.

**4. No modifiques la variable de control en for**

Puede provocar comportamientos inesperados y errores difíciles de detectar.

**5. Comenta tu código**

Explica el propósito de tus condiciones y ciclos complejos. Facilita entenderlo después (o para otros).

**6. Evita cálculos innecesarios**

Coloca las condiciones antes de ejecutar operaciones costosas cuando sea posible.

Consejo general: Mantén tu código claro, ordenado y bien comentado. ¡Tu futuro tú (y tus compañeros) te lo agradecerán!



1 ¿QUÉ ES UN SCRIPT?

Un script es un archivo .m que contiene una secuencia de instrucciones que se ejecutan en el orden en que aparecen.

- ✓ No inicia con la palabra **function**.
- ✓ Trabaja directamente con las variables del Workspace.
- ✓ Se usa para organizar y ejecutar tareas, cálculos, gráficas, etc.
- ✓ Se ejecuta escribiendo el nombre del archivo (sin la extensión .m).

EJEMPLO: archivo ejemplo1.m

```
% Script: ejemplo1.m
clc; clear;
A = rand(3);
b = [1; 2; 3];
x = A\b;
disp('Solución x = ');
disp(x)
```

El script comparte el **Workspace**: las variables creadas quedan disponibles al terminar su ejecución.

2 ¿QUÉ ES UNA FUNCIÓN?

Una función es un bloque de código que realiza una tarea específica. Puede recibir datos de entrada, procesarlos y devolver resultados.

EJEMPLO: archivo cuadratica.m

```
function x = cuadratica(a,b,c) % Función principal
    d = b^2 - 4*a*c;
    x1 = (-b + sqrt(d))/(2*a);
    x2 = (-b - sqrt(d))/(2*a);
    x = [x1, x2];
end
```

Entrada
a, b, c

Proceso
cálculos internos

Salida
x (vector 1x2)

3 REGLA ESENCIAL DEL NOMBRE

La función principal **DEBE** tener el mismo nombre que el archivo .m

✓ Correcto:

```
function y = miFuncion(x)
...
end
```

Archivo: **miFuncion.m**

✗ Incorrecto:

```
function y = miFuncion(x)
...
end
```

Archivo: **otraCosa.m**

Octave busca la función principal usando el nombre del archivo. Si no coinciden, la función puede no ser encontrada.

4 FUNCIÓN PRINCIPAL Y FUNCIONES AUXILIARES

Un archivo puede contener una función principal y varias funciones auxiliares (locales). La función principal siempre debe ir primero.

EJEMPLO: archivo cuadratica.m

```
% ===== FUNCIÓN PRINCIPAL (debe ir primero) =====
function x = cuadratica(a,b,c) % llama a función auxiliar
    d = discriminante(a,b,c);
    x1 = (-b + sqrt(d))/(2*a);
    x2 = (-b - sqrt(d))/(2*a);
    x = [x1, x2];
end

% ===== FUNCIÓN AUXILIAR (puede ir después) =====
function d = discriminante(a,b,c)
    d = b^2 - 4*a*c;
end
```

- ✓ La función principal aparece primero en el archivo.
- ✓ Su nombre coincide con el nombre del archivo.
- ✓ Las funciones auxiliares se escriben debajo.
- ✓ Las funciones auxiliares solo son visibles dentro del archivo y son usadas desde la función principal u otras auxiliares.
- ✓ Cada función tiene su propio espacio de variables locales.

5 DIFERENCIAS ENTRE SCRIPT Y FUNCIÓN

Aspecto	Script	Función
Propósito	Ejecuta una secuencia de instrucciones.	Realiza una tarea específica.
Entradas	No tiene entradas formales. Usa variables definidas en el Workspace.	Puede recibir una o más entradas (argumentos).
Salidas	No tiene salidas formales. Los resultados se muestran o guardan en el Workspace.	Puede devolver una o más salidas (resultados).
Ámbito de las variables (Workspace)	Usa el Workspace principal. Las variables quedan disponibles al finalizar.	Tiene un espacio local. Las variables internas NO quedan en el Workspace.
Inicio del archivo	No inicia con function .	Debe iniciar con la palabra function .
Reutilización	Se usa para tareas completas o pruebas.	Se diseña para reutilizar código en distintos programas.
Ejemplo de llamada	<code>ejemplo1</code> (escribir nombre del archivo)	<code>x = cuadratica(1,2,-3)</code> (con argumentos y/o capturando salidas)

6 ¿CÓMO LLAMAR UNA FUNCIÓN?

Con argumentos de entrada:

```
x = cuadratica(1, 2, -3);
disp(x) % devuelve vector 1x2
```

Sin argumentos de entrada:

```
mostrarMenu(); % función que no devuelve salidas
```

Con múltiples salidas:

```
[r1, r2] = operaciones(5, 3);
disp(r1); disp(r2); % dos resultados
```

¿Cuántas entradas y salidas puede tener una función?

Entradas	Salidas	Ejemplo de definición	Ejemplo de llamada
Ninguna	Ninguna	<code>function ejemplo()</code>	<code>ejemplo()</code>
Una	Una	<code>function y = f(x)</code>	<code>y = f(2)</code>
Varias	Una	<code>function z = f(x,y)</code>	<code>z = f(1,2)</code>
Varias	Varias	<code>function [x1,x2] = f(a,b,c)</code>	<code>[x1,x2] = f(1,2,3)</code>

Si pides más salidas de las que devuelve la función → error.

7 ANATOMÍA DE UNA FUNCIÓN

```
function [sal1, sal2] = nombreFuncion(entrada1, entrada2)
```

Palabra reservada
Variables de salida (puede ser 0, 1 o varias)
Nombre de la función (igual al archivo .m)
Variables de entrada (puede ser 0, 1 o varias)

Los nombres de variables de salida y entrada son opcionales, salvo que se usen dentro de la función.

8 VARIABLES LOCALES

Las variables creadas dentro de una función son **locales**: existen solo durante su ejecución.

```
function y = ejemplo(x)
    auxiliar = x^2; % variable local
    y = auxiliar + 1;
end
```

Después de llamar a la función, la variable **auxiliar NO** estará disponible en el Workspace.

9 COMENTARIOS Y AYUDA

Los comentarios iniciales sirven como documentación. Pueden consultarse con `help nombreFuncion`.

```
function x = cuadratica(a,b,c)
% CUADRATICA Calcula las raíces de ax^2 + bx + c = 0.
% Entrada:
%   a, b, c : coeficientes (a ≠ 0)
% Salida:
%   x : vector 1x2 con las dos soluciones
% -----
... código ....
end
```

```
>> help cuadratica % muestra la ayuda en la ventana de comandos
```

10 BUENAS PRÁCTICAS

1. Usa nombres descriptivos para variables, scripts y funciones.
2. Cada función debe tener una única tarea bien definida.
3. Documenta entradas, salidas y lo que hace la función.
4. Evita `clc`, `clear` y `close all` dentro de funciones.
5. Usa funciones auxiliares para dividir problemas complejos.
6. Indenta tu código y usa comentarios para mejorar la legibilidad.

ERRORES FRECUENTES

- ✗ Guardar la función principal con un nombre diferente al del archivo.
- ✗ Colocar una función auxiliar antes de la función principal.
- ✗ Olvidar pasar los argumentos requeridos a la función.
- ✗ Solicitar más salidas de las que la función devuelve.
- ✗ Usar una variable creada dentro de la función fuera de ella.
- ✗ Olvidar cerrar la función con `end`.
- ✗ No documentar entradas y salidas.





1 RESOLVER SISTEMAS DE ECUACIONES LINEALES

El problema de álgebra lineal más común es resolver sistemas de la forma $Ax = b$.

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

Ejemplo en Octave

```
A = [ 5 1 0;  
      1 5 1;  
      0 1 5];
```

```
b = [-6;  
     -7;  
     -6];
```

```
x = A\b    % solución del sistema
```

¿Qué hace A\b?

- ✓ Calcula el vector x que satisface $Ax = b$.
- ✓ Utiliza algoritmos numéricos estables y eficientes.
- ✓ Es el método recomendado (no es necesario calcular la inversa de A).



Evita usar `inv(A)*b`:
es menos preciso y
más costoso.

2 DIAGNÓSTICO DE MATRICES

Antes de resolver un sistema, es útil analizar la matriz A .

Función	Sintaxis	¿Para qué sirve?	Interpretación rápida
<code>det(A)</code>	<code>det(A)</code>	Determinante (de matrices cuadradas).	Si $\det(A) = 0 \rightarrow$ A es singular (no tiene inversa).
<code>rank(A)</code>	<code>rank(A)</code>	Número de filas o columnas linealmente independientes.	Si $\text{rank}(A) = n \rightarrow$ A tiene rango completo.
<code>cond(A)</code>	<code>cond(A)</code>	Condición numérica (estimación).	$\text{cond}(A)$ grande $\rightarrow$ el sistema puede ser mal condicionado.
<code>norm(A)</code>	<code>norm(A)</code>	Norma (tamaño) de la matriz.	Mide la magnitud global de A .



Usar estas funciones ayuda a decidir si un sistema está bien planteado y si la solución será confiable.

3 FACTORIZACIONES DE MATRICES

Octave incluye varias funciones para factorizar matrices. Estas descomposiciones son la base de muchos métodos numéricos que se verán más adelante en el curso.

LU (Eliminación Gaussiana)

Decompone A en L (triangular inferior) y U (triangular superior) con posible permutación de filas.

```
[L,U,P] = lu(A)
```

$$P \cdot A = L \cdot U$$

Útil para resolver sistemas
y calcular determinantes.

QR (Ortogonal - Triangular)

Decompone A en Q (ortogonal) y R (triangular superior).

```
[Q,R] = qr(A)
```

$$A = Q \cdot R$$

Útil para mínimos cuadrados
y problemas de estabilidad numérica.

Cholesky (para matrices SPD)

Para matrices simétricas definidas positivas (SPD): $A = R^T \cdot R$.

```
R = chol(A)
```

$$A = R^T \cdot R$$

Más eficiente que LU cuando A
es simétrica definida positiva.

★ IMPORTANTE

Estas funciones devuelven factores que podrás usar para resolver sistemas, calcular determinantes, inversas (cuando sea necesario) y mucho más.

Se estudiarán en detalle en el curso.

4 BUENAS PRÁCTICAS

- ✓ Resolver sistemas usando `A\b`.
`x = A\b`
- ✓ Verificar dimensiones antes de operar.
`size(A)`
`size(b)`
- ✓ Verificar el resultado calculando el residuo.
`norm(A*x - b)`
- ✓ Usar nombres descriptivos.
`A`, `b`, `x`
- ✓ Revisar la condición de A si el sistema da resultados inesperados.
`cond(A)`
- ✓ Comentar tu código.
`% Resuelve Ax = b`
`x = A\b;`

5 ERRORES FRECUENTES

- ✗ Calcular la inversa y luego multiplicar.
`x = inv(A)*b`
Menos preciso, más costoso.
- ✗ Multiplicar matrices con dimensiones incompatibles.
`A * B`
Verifica `size(A)` y `size(B)`.
- ✗ Olvidar que `A\b` requiere que `size(A,1) == length(b)`.
`size(A,1) != length(b)`
El número de filas de A debe igualar el tamaño de b .
- ✗ Confiar solo en `det(A)`.
`det(A) ≈ 0`
`det` pequeño no siempre implica mal condición.
- ✗ No verificar el resultado obtenido.
(sin verificar)
Calcula `norm(A*x-b)`.
- ✗ Usar `chol(A)` en matrices que no son simétricas definidas positivas.
`chol(A)`
Puede generar error o resultados incorrectos.

6 FUNCIONES QUE USARÁS DURANTE EL CURSO

<code>det</code>	<code>rank</code>	<code>norm</code>	<code>cond</code>	<code>lu</code>	<code>qr</code>	<code>chol</code>	<code>eig</code>
Determinante de matrices cuadradas.	Rango de la matriz.	Norma de la matriz o vector.	Condición numérica (estimación).	Factorización LU.	Factorización QR.	Factorización de Cholesky.	Valores y vectores propios.



**1 FUNCIONES ANÓNIMAS (EL PUNTO DE PARTIDA)**

Las funciones anónimas se crean con @ y representan expresiones matemáticas que pueden evaluarse y usarse en otras funciones.

```
f = @(x) x.^2 - 4;      % f(x) = x^2 - 4
f(3)
ans = 5
```

Ejemplos

```
g = @(x,y) x.^2 + y.^2;
h = @(x) sin(x)./x;
p = @(t) exp(-t).*cos(t);
```

Evaluación

```
g(1,2)      % 5
h(linspace(0,1,5))
p([0 0.5 1])
```

TIP: Use .*, ./ y .^ cuando la función reciba vectores.

2 ¿QUÉ FUNCIÓN USAR?

Función	¿Qué resuelve?	Entrada principal	Salida
fzero	Una ecuación $f(x) = 0$	Función y aproximación inicial o intervalo	Raíz r tal que $f(r) \approx 0$
fsolve	Sistema de ecuaciones no lineales $F(x) = 0$	Función vectorial y aproximación inicial	Vector x que satisface $F(x) \approx 0$
fminsearch	Mínimo local de una función	Función y punto inicial	Punto $x_{\min}$ del mínimo aproximado
fminbnd	Mínimo en un intervalo $[a, b]$	Función y extremos del intervalo	Punto $x_{\min}$ en $[a, b]$ que minimiza $f(x)$

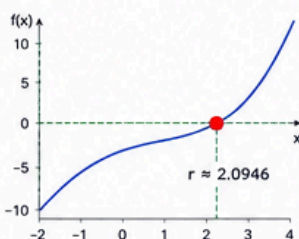
3 ENCONTRAR RAÍCES (fzero)

Busca una raíz de la ecuación $f(x) = 0$.

```
f = @(x) x.^3 - 2*x - 5;
r = fzero(f,2);
r =
2.0946
```

Interpretación

fzero encuentra el valor r donde $f(r) = 0$ usando una aproximación inicial (2).



TIP: Grafique la función para elegir una buena aproximación inicial.

4 RESOLVER SISTEMAS NO LINEALES (fsolve)

Resuelve sistemas de ecuaciones no lineales $F(x) = 0$.

Ejemplo: sistema 2x2

$$\begin{cases} x^2 + y^2 = 5 \\ x - y = 1 \end{cases}$$

```
F = @(x) [ x(1)^2 + x(2)^2 - 5;
           x(1) - x(2) - 1 ];
x0 = [1; 1];
sol = fsolve(F, x0);
sol =
1.6180
0.6180
```

Verificación

```
F(sol) =
-8.88e-16
-2.22e-16      % ≈ 0
```

TIP: x debe ser un vector columna con las variables del sistema.

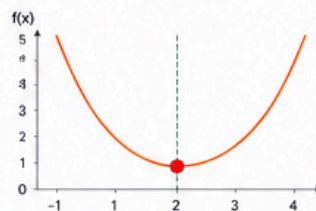
5 ENCONTRAR MÍNIMOS (fminsearch)

Busca un mínimo local de una función sin necesidad de derivadas.

```
f = @(x) (x-2).^2 + 1;
xmin = fminsearch(f,0);
xmin =
2.0000
f(xmin) =
1.0000
```

Interpretación

El mínimo de f ocurre en $x = 2$ con valor $f(2) = 1$.



TIP: Necesita una aproximación inicial (no un intervalo).

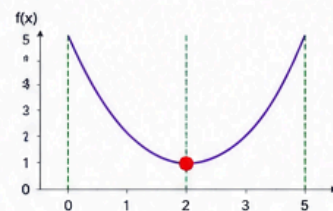
6 MÍNIMO EN UN INTERVALO (fminbnd)

Busca el mínimo de $f(x)$ dentro de un intervalo $[a, b]$.

```
f = @(x) (x-2).^2 + 1;
xmin = fminbnd(f,0,5);
xmin =
2.0000
f(xmin) =
1.0000
```

Interpretación

Encuentra el mínimo en el intervalo $[0, 5]$.



TIP: Si el mínimo está fuera del intervalo, la solución será un extremo.

7 OTRAS HERRAMIENTAS QUE USARÁS DURANTE EL CURSO

Integración numérica
Calcula aproximaciones de integrales definidas.

```
quad      % integración adaptativa
quadgk    % Gauss-Kronrod
integral   % interfaz moderna
```

Ejemplo:

```
f = @(x) exp(-x.^2);
quad(f, 0, 1)
```

Ecuaciones diferenciales ordinarias (EDO)
Resuelve $y' = f(t,y)$.

```
ode45     % Runge-Kutta (4,5)
lsode     % método BDF / Adams
```

Ejemplo:

```
f = @(t,y) -2*y;
ode45(f, [0 5], 1)
```

Ajuste de datos (polinomios)
Ajusta y evalúa polinomios a partir de datos.

```
polyfit   % ajuste polinomial
polyval   % evaluar polinomios
polyder   % derivada polinom.
```

Ejemplo:

```
x = 0:4; y = x.^2 + 1;
p = polyfit(x,y,2);
polyval(p, 2.5)
```

Raíces de polinomios
Encuentra todas las raíces de un polinomio.

```
roots     % raíces de polinomios
```

Ejemplo:

```
p = [1 0 -4];      % x^2 - 4
roots(p)
```

Optimización general
Otras funciones de optimización que se estudiarán.

```
fminsearch % sin restricciones
fminbnd    % en un intervalo
fminunc    % con gradiente
```

Ejemplo:

```
f = @(x) (x-1).^2;
fminunc(f, 0)
```

Estas funciones se estudiarán en detalle en las unidades de: Integración, Ecuaciones Diferenciales, Interpolación, Regresión y Optimización.

8 BUENAS PRÁCTICAS

- ✓ Grafique la función antes de usar fzero o fminbnd.
- ✓ Elija una buena aproximación inicial para fzero, fminsearch, fsolve.
- ✓ Verifique el resultado sustituyendo en la ecuación o evaluando la función.
- ✓ Use operadores elemento a elemento (.*, ./, .^)
- ✓ si su función recibe vectores.
- ✓ Para fsolve, devuelva un vector con una ecuación por componente.

9 ERRORES FRECUENTES

✗ Olvidar el operador @ al crear la función anónima.	$f = (x) x^2$	Debe ser: $f = @(x) x.^2$
✗ Usar ^ en lugar de .^ cuando la función recibe vectores.	$f = @(x) x^2$	Debe ser: $f = @(x) x.^2$
✗ Elegir una aproximación inicial muy alejada de la solución.	<code>fzero(f, 1000)</code>	La función puede no converger.
✗ En fsolve devolver un escalar cuando el sistema tiene varias ecuaciones.	$F = @(x) x(1)^2 + x(2)^2 - 5$	Debe devolver un vector.
✗ No revisar si la función convergió.	<code>[x, fval, info] = fzero(f, 2)</code>	Use info o verifique el residuo.

info > 0 → Convergió

info = 0 → Iteraciones máximas

info < 0 → No convergió





1 CARGAR EL PAQUETE SIMBÓLICO

Debe cargarse al iniciar cada nueva sesión.

```
pkg load symbolic % cargar el paquete
```



Si no está instalado:

```
pkg install -forge symbolic % instalar (una sola vez)
```

2 CREAR VARIABLES SIMBÓLICAS

Las variables simbólicas representan cantidades matemáticas exactas.

```
syms x % una variable
syms x y % dos variables
syms a b c real % variables reales
x = sym('x'); % otra forma
```



Una variable simbólica no tiene un valor numérico hasta que se le sustituye uno.

3 CREAR EXPRESIONES SIMBÓLICAS

Se construyen expresiones matemáticas exactas.

```
syms x
f = exp(x) + 4*x; % f(x) = e^x + 4x
syms x y
g = x^2 + x*y + y^2; % g(x,y) = x^2 + xy + y^2
```

Diferencia importante

```
f_sym = exp(x) + 4*x; % simbólica
f_num = @(x) exp(x) + 4*x; % numérica
```

4 PASAR DE TEXTO A SIMBÓLICO

Convierte una cadena de texto en una expresión simbólica.

```
f_text = 'exp(x)+4*x'; % texto
syms x % definir variables
f_sym = sym(f_text); % simbólica
```

Texto
'exp(x)+4*x'

→ sym →

Expresión
simbólica



La cadena debe usar los mismos nombres de variables simbólicas que haya definido.

5 PASAR DE TEXTO A FUNCIÓN NUMÉRICA

Convierte una cadena de texto en una función numérica (función anónima).

```
f_text = 'exp(x)+4*x'; % texto
f_num = str2func(['@(x)' f_text]); % función numérica
f_num(2) ans = 14.7781
```

Texto
'exp(x)+4*x'

→ str2func →

Función
numérica



La función numérica se usa para evaluaciones repetidas y en métodos numéricos.

6 PASAR DE SIMBÓLICO A NUMÉRICO

Convierte una expresión simbólica en una función numérica.

```
syms x
f_sym = exp(x) + 4*x;
f_num = matlabFunction(f_sym); % función numérica
f_num(2) ans = 14.7781
```

Con derivadas

```
df_sym = diff(f_sym);
f_df = matlabFunction(df_sym);
f_df(2) ans = 6.7781
```

Simbólico

→ matlabFunction →

Función
numérica



Úselo para evaluaciones repetidas o en métodos numéricos.

7 DERIVADAS CON diff

Calcula derivadas simbólicas.

```
syms x
f = sqrt(sin(x)*log(x));
df = diff(f); % primera derivada
d2f = diff(f,2); % segunda derivada
```

Con dos variables

```
syms x y
g = x^2*y + sin(x*y);
gx = diff(g,x); % ∂g/∂x
gy = diff(g,y); % ∂g/∂y
```

Uso general

```
diff(f) % respecto de la variable definida
diff(f,2) % segunda derivada
diff(f,x) % especifica variable
diff(f,x,2) % segunda derivada respecto de x
```

8 SIMPLIFICAR, EXPANDIR Y FACTORIZAR

simplify Simplifica una expresión.

```
syms x
f = sin(x)^2 + cos(x)^2;
simplify(f) ans = 1
```

expand Expande productos y potencias.

```
g = (x+2)^3;
expand(g) ans = x^3 + 6*x^2 + 12*x + 8
```

factor Factoriza polinomios.

```
h = x^2 - 5*x + 6;
factor(h) ans = (x-2)*(x-3)
```

★ **simplify** reduce, **expand** desarrolla, **factor** factoriza.

9 SUSTITUCIÓN Y EVALUACIÓN CON subs

Sustituye valores en expresiones simbólicas.

Una variable

```
syms x
f = x^2 + 1;
subs(f,x,3) ans = 10
```

Varias variables

```
syms x y
g = x^2 + y;
subs(g,[x y],[2 5]) ans = 9
```



Use **subs** para sustituciones simbólicas o evaluaciones ocasionales.



Para muchas evaluaciones numéricas, convierta con **matlabFunction**.

10 CONVERTIR A NÚMERO CON double

Convierte un resultado simbólico constante a número de punto flotante.

```
syms x
r = subs(sqrt(x),x,2); % √2
r ans = sqrt(2)
```

```
double(r) ans = 1.414213562373095
```



double convierte valores simbólicos constantes a números aproximados.

11 RESOLVER ECUACIONES CON solve

Ecuación

```
syms x
sol = solve(x^2 - 5*x + 6 == 0, x)
sol =
2
3
```

Sistema de ecuaciones

```
syms x y
sol = solve([x + y == 5, x - y == 1], [x,y])
sol.x = 3
sol.y = 2
```

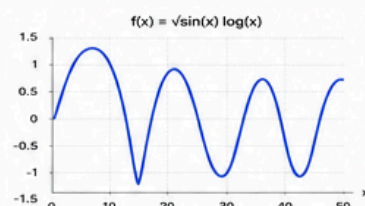


Útil para obtener soluciones exactas cuando es posible.

12 GRAFICAR EXPRESIONES SIMBÓLICAS

Grafica funciones definidas simbólicamente.

```
syms x
f = sqrt(sin(x)*log(x));
fplot(f,[1 50]);
grid on
```



También puede usar **ezplot** en algunas versiones de Octave: **ezplot(f,[1 50])**

13 RESUMEN DE COMANDOS CLAVE

Comando	¿Qué hace?	Ejemplo	Resultado
syms	Crea variables simbólicas	syms x y	variables simbólicas x, y
sym(texto)	Texto → expresión simbólica	sym('exp(x)+4*x')	exp(x) + 4*x
str2func	Texto → función numérica	str2func(['@(x)exp(x)+4*x'])	función numérica
diff	Deriva expresiones	diff(x^2,2)	2
simplify	Simplifica expresiones	simplify(sin(x)^2+cos(x)^2)	1
expand	Expande productos/potencias	expand((x+1)^3)	x^3 + 3*x^2 + 3*x + 1
factor	Factoriza polinomios	factor(x^2-5*x+6)	(x-2)*(x-3)
subs	Sustituye valores	subs(x^2+x, x, 3)	12
solve	Resuelve ecuaciones	solve(x^2-5*x+6==0)	[2; 3]
double	Simbólico → número	double(sqrt(2))	1.4142
fplot	Grafica expresiones	fplot(sin(x), [0 2*pi])	Gráfica

14 BUENAS PRÁCTICAS

- ✓ Cargue el paquete **symbolic** antes de usar sus comandos.
- ✓ Use **simbólico** para derivar, simplificar o manipular expresiones exactas.
- ✓ Especifique la variable correcta en **diff** cuando haya varias variables.
- ✓ Convierta a función numérica (**matlabFunction**) antes de usar expresiones en ciclos o métodos numéricos.
- ✓ Use **subs** para sustituciones simbólicas o evaluaciones ocasionales.
- ✓ Use **double** solo cuando ya no necesite la representación exacta.

15 ERRORES FRECUENTES

- ✗ Olvidar ejecutar **pkg load symbolic**.
- ✗ Confundir **sym('exp(x)+4*x')** con **str2func**.
- ✗ Usar **subs** en un bucle o para muchas evaluaciones numéricas.
- ✗ Olvidar el operador **.** en lugar de **^** cuando se esperan vectores.
- ✗ Usar **double** sobre una expresión que todavía contiene variables.
- ✗ No indicar la variable correcta en una derivada parcial.





1 CREAR LOS DATOS

Defina los vectores x (puntos) y y (valores).

```
x = linspace(0,2*pi,200); % 200 puntos entre 0 y 2π
y = sin(x);                % valores de la función
```

x : vector de puntos en el eje X

y : vector de valores correspondientes



TIP: Use suficientes puntos para obtener una curva suave (50 - 500 puntos es recomendable).

2 GRAFICAR

El comando básico para graficar.

```
plot(x,y) % gráfica por defecto
```



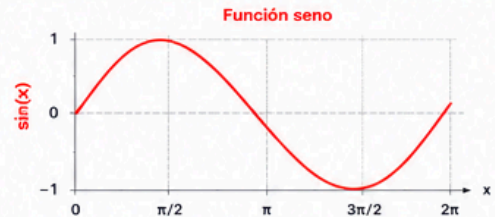
`plot(x,y)` dibuja y versus x .
Si x no se especifica, Octave usa 1,2,3,...,n.

3 PERSONALIZAR LA GRÁFICA

Mejore la presentación de su gráfico.

```
plot(x,y,'r','LineWidth',2)
title('Función seno')
xlabel('x')
ylabel('sin(x)')
grid on
```

—	'r'	Color rojo
—	LineWidth	Grosor de línea
 	title	Título del gráfico
x	xlabel	Etiqueta del eje X
y	ylabel	Etiqueta del eje Y
	grid on	Cuadrícula

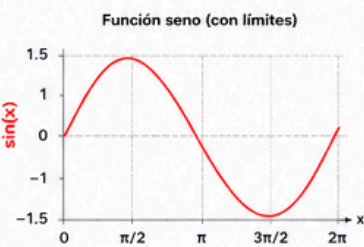


Puede usar otros colores y estilos (ver sección 6).

4 LÍMITES DE LOS EJES

Controle la región visible del gráfico.

```
xlim([0 2*pi]) % límites en X
ylim([-1.5 1.5]) % límites en Y
```

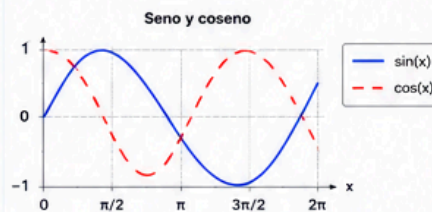


`xlim([a b])` define el intervalo visible del eje X.
`ylim([c d])` define el intervalo visible del eje Y.

5 DIBUJAR VARIAS FUNCIONES

Use `hold on` para agregar más curvas.

```
plot(x,sin(x),'b','LineWidth',2)
hold on
plot(x,cos(x),'r--','LineWidth',2)
hold off
legend('sin(x)','cos(x)')
```



- ✓ `hold on` : mantiene el gráfico activo para agregar más.
- ✓ `hold off` : termina la superposición de gráficos.
- ✓ `legend` : muestra la leyenda de las curvas.

6 COLORES, ESTILOS Y MARCADORES

COLORES BÁSICOS

■	■	■	■	■	■	■
rojo	azul	verde	negro	magenta	cian	amarillo

ESTILOS DE LÍNEA

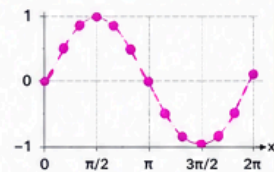
—	línea continua
--	discontinua
...	punteada
-.-	punto-guion

MARCADORES

○	círculo
*	estrella
s	cuadrado
A	triángulo
+	cruz

EJEMPLO COMBINADO

```
plot(x,sin(x),'m--o','LineWidth',2,
'MarkerFaceColor','m')
```



Puede combinar color + estilo + marcador.

7 GUARDAR LA FIGURA

Guarde su gráfico en un archivo.

Como imagen (PNG, JPG, etc.)

```
saveas(gcf,'grafica.png') % usa la figura actual
```

Como archivo PDF

```
print('grafica.pdf','-dpdf')
```



Asegúrese de que la figura se vea como desea antes de guardarla.

8 FUNCIONES ÚTILES

Comandos que facilitan su trabajo.

```
figure % abre una nueva ventana
clf % limpia la figura actual
close % cierra la ventana actual
close all % cierra todas las ventanas
axis equal % misma escala en X e Y
axis square % escala igual y ejes cuadrados
```



Úselos para organizar y mejorar sus gráficos.

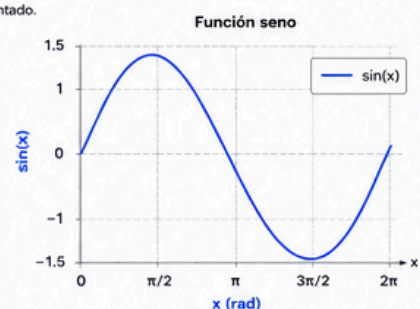
9 EJEMPLO COMPLETO

Plantilla de un gráfico 2D bien presentado.

```
x = linspace(0,2*pi,200);
y = sin(x);
figure
plot(x,y,'b','LineWidth',2)
title('Función seno')
xlabel('x (rad)')
ylabel('sin(x)')
xlim([0 2*pi])
ylim([-1.5 1.5])
grid on
legend('sin(x)')
```



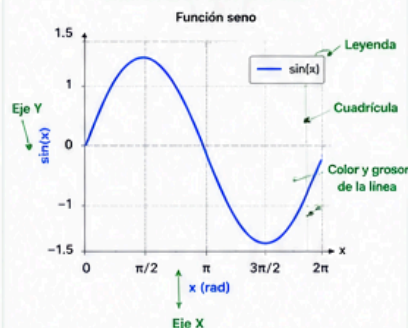
Este es un esquema que puede reutilizar siempre.



10 BUENAS PRÁCTICAS

- ✓ Use suficientes puntos (`linspace`) para una curva suave.
- ✓ Siempre etiquete ambos ejes (`xlabel`, `ylabel`).
- ✓ Agregue un título descriptivo (`title`).
- ✓ Active la cuadrícula (`grid on`) para facilitar la lectura.
- ✓ Use colores y estilos con buen contraste.
- ✓ Cuando haya varias curvas, incluya `legend`.
- ✓ Ajuste los límites de los ejes (`xlim`, `ylim`) para mostrar la región de interés.
- ✓ Use `LineWidth` adecuado para mejorar la visibilidad.

ANATOMÍA DE UNA GRÁFICA



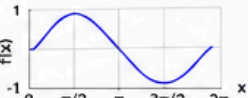
11 ERRORES FRECUENTES

✗ No especificar x en <code>plot</code> .	<code>plot(y)</code>	Solución: <code>plot(x,y)</code>
✗ Olvidar <code>hold on</code> al graficar varias curvas.	<code>plot(x,sin(x))</code> <code>plot(x,cos(x))</code>	Solución: use <code>hold on</code> antes de la segunda gráfica.
✗ Usar muy pocos puntos.	<code>x = linspace(0,2*pi,5)</code>	Solución: use más puntos, por ejemplo 100 o 200.
✗ No etiquetar los ejes ni el título.	<code>plot(x,y)</code>	Solución: agregue <code>xlabel</code> , <code>ylabel</code> y <code>title</code> .
✗ Límites inadecuados.	<code>ylim([0 1])</code>	Solución: revise los límites correctos.
✗ Colores o estilos poco claros.	<code>plot(x,y,'y:')</code>	Solución: use colores contrastantes y mayor <code>LineWidth</code> .

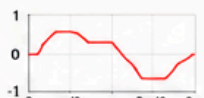




BUENAS PRÁCTICAS

1	Organización del código	Inicie su sesión limpia y organice el código por secciones. <pre>clear %% Datos clc %% Cálculos close all %% Resultados %% Gráficas</pre>
2	Nombres de variables	Use nombres descriptivos y significativos. <pre>numero_iteraciones = 50; error_relativo = 1e-6; matriz_coeficientes = A;</pre>
3	Uso del punto y coma (;)	Use ; para evitar imprimir resultados innecesarios en la consola. <pre>A = rand(1000); % no se imprime b = 2*pi; c = A*b;</pre>
4	Operaciones elemento a elemento	Use .*, ./ y .^ para operaciones elemento a elemento. <pre>x = [1 2 3 4]; y = x.^2; % [1 4 9 16] z = x.*sin(x); w = x./(1 + x);</pre>
5	Comparaciones y condicionales	Use == para comparar y considere tolerancias para números reales. <pre>if abs(x - y) < 1e-8 disp('Son iguales') end</pre>
6	Dimensiones compatibles	Verifique dimensiones antes de operar. <pre>size(A) % p x q size(B) % q x r C = A*B; % correcto</pre>
7	Preasignación de memoria	Preasigne memoria en vectores y matrices. <pre>n = 10000; y = zeros(1,n); for k = 1:n y(k) = k^2; end</pre>
8	Sistemas lineales	Use el operador \ para resolver $Ax = b$. <pre>x = A\b; % Más estable y eficiente</pre>
9	Funciones y archivos	El nombre del archivo debe coincidir con la función principal y esta debe ir primero. <pre>function y = cuadrado(x) y = x.^2; end % Archivo: cuadrado.m</pre>
10	Ciclos	Asegúrese de que la condición de parada pueda alcanzarse. <pre>while error > tolerancia ... error = nuevo_error; end</pre>
11	Gráficas	Siempre etiquete ejes, agregue título y cuadrícula, y use suficientes puntos. <pre>plot(x,y,'LineWidth',2,'b') xlabel('x') ylabel('f(x)') title('MI gráfica') grid on</pre> 

ERRORES FRECUENTES

1	Código desordenado, sin secciones, con variables antiguas en memoria.	<pre>A = rand(5); x = 1; y = 2; plot(A) B = A*x + y % (muchas líneas sin separación)</pre>	• Difícil de leer. • Difícil de depurar. • Resultados inesperados por variables previas.
2	Use nombres poco claros o que sobrescriben funciones de Octave.	<pre>a = 50; x1 = 1e-6; sum = 5; % X sobrescribe la función sum() plot = 3; % X sobrescribe la función plot()</pre>	• Código confuso. • Errores difíciles de encontrar. • Pérdida de funcionalidad.
3	Omitir ; puede llenar la consola con resultados extensos.	<pre>A = rand(1000) b = 2*pi c = A*b</pre> ans = Column 1 through 6 0.8147 0.9058 0.1270 0.9134 0.6324 0.0975 0.9134 0.2785 0.5469 0.9575 0.9649 0.1577 ... (muchos más números)	• La consola se llena de información innecesaria.
4	Usar operadores matriciales en lugar de elemento a elemento.	<pre>x = [1 2 3 4]; y = x^2; % X error (potencia matricial) z = x*sin(x); % X error w = x/(1 + x); % X error</pre>	• Errores de dimensiones. • Resultados incorrectos.
5	Usar = en lugar de == o comparar reales sin tolerancia.	<pre>if x = 0 % X asigna 0 a x disp('x es cero') end</pre>	• La condición siempre es verdadera o falsa. • Errores lógicos muy comunes.
6	Operaciones con dimensiones incompatibles.	<pre>% A es p x q % B es m x n C = A*B; % X error de dimensiones</pre>	• Errores en tiempo de ejecución. • Resultados indefinidos.
7	No preasignar y crecer el vector en el ciclo.	<pre>n = 10000; y = []; for k = 1:n % X crecimiento dinámico lento y(k) = k^2; end</pre>	• Código más lento. • Mayor uso de memoria.
8	Usar inv(A) es menos preciso y menos eficiente.	<pre>x = inv(A)*b; % X No recomendado</pre>	• Menor precisión numérica. • Más costoso computacionalmente.
9	Nombre de archivo distinto o mal estructurado.	<pre>function y = cuadrado(x) y = x.^2; end % Archivo: otro_nombre.m % X incorrecto</pre>	• Octave no encontrará la función. • Errores al ejecutar.
10	La condición nunca cambia (ciclo infinito).	<pre>while error > tolerancia ... % error no se actualiza end</pre>	• El programa queda detenido en un ciclo infinito.
11	Gráfica poco informativa.	<pre>plot(y) title('grafica')</pre> 	• No se sabe qué se grafica. • Sin etiquetas ni contexto. • Dificulta la interpretación.

ANTES DE EJECUTAR, REVISE

- ✓ ¿Las variables tienen nombres claros?
- ✓ ¿Usó .*, ./, .^ cuando corresponde?
- ✓ ¿Agregó ; en resultados extensos?
- ✓ ¿La condición de parada puede alcanzarse?
- ✓ ¿Las gráficas tienen título y etiquetas?
- ✓ ¿El archivo tiene el mismo nombre que la función principal?
- ✓ ¿Los resultados son razonables?



FORMULARIO DE COMANDOS

GNU Octave - Parte I

Resumen de los comandos y funciones esenciales utilizados durante el curso.



GNU Octave

Lenguaje libre para el cómputo numérico y el análisis de problemas de ingeniería.

1 ENTORNO DE TRABAJO

Función	Descripción
<code>clc</code>	Limpia la ventana de comandos.
<code>clear</code>	Elimina las variables del Workspace.
<code>close</code>	Cierra la figura actual.
<code>close all</code>	Cierra todas las figuras abiertas.
<code>who</code>	Lista las variables existentes.
<code>whos</code>	Muestra información detallada de las variables.
<code>help</code>	Muestra la ayuda de una función.
<code>pkg list</code>	Lista los paquetes instalados.
<code>pkg load symbolic</code>	Carga el paquete simbólico.



Use `help nombre_comando` para más información.

2 CREACIÓN DE VECTORES Y MATRICES

Función	Descripción
<code>zeros</code>	Crea una matriz de ceros.
<code>ones</code>	Crea una matriz de unos.
<code>eye</code>	Crea una matriz identidad.
<code>rand</code>	Genera números aleatorios uniformes.
<code>randi</code>	Genera enteros aleatorios.
<code>linspace</code>	Genera valores igualmente espaciados.
<code>logspace</code>	Genera valores espaciados logarítmicamente.



Ejemplo: `A = zeros(3,4)` crea una matriz de 3 filas y 4 columnas con ceros.

3 MANIPULACIÓN DE MATRICES

Función	Descripción
<code>size</code>	Devuelve el tamaño de una matriz.
<code>length</code>	Devuelve la longitud de un vector.
<code>numel</code>	Devuelve el número total de elementos.
<code>sum</code>	Calcula la suma de los elementos.
<code>mean</code>	Calcula el promedio.
<code>max</code>	Devuelve el valor máximo.
<code>min</code>	Devuelve el valor mínimo.
<code>diag</code>	Extrae o crea diagonales.
<code>reshape</code>	Cambia la forma de una matriz.
<code>sort</code>	Ordena los elementos.
<code>flip</code>	Invierte el orden de un vector o matriz.
<code>dot</code>	Calcula el producto punto.



Muchas funciones pueden operar por filas (`dim=1`) o por columnas (`dim=2`).

4 ÁLGEBRA LINEAL

Función	Descripción
<code>det</code>	Calcula el determinante de una matriz.
<code>rank</code>	Calcula el rango de una matriz.
<code>cond</code>	Calcula el número de condición.
<code>norm</code>	Calcula la norma de un vector o matriz.
<code>lu</code>	Realiza la factorización LU.
<code>qr</code>	Realiza la factorización QR.
<code>chol</code>	Realiza la factorización de Cholesky.
<code>eig</code>	Calcula valores y vectores propios.



Úselas para analizar propiedades y resolver sistemas lineales.

5 OPERADORES ÚTILES

Operador	Descripción
<code>:</code>	Genera rangos o selecciona índices.
<code>.'</code>	Transpuesta (sin conjugación).
<code>'</code>	Transpuesta conjugada.
<code>.*</code>	Multiplicación elemento a elemento.
<code>./</code>	División elemento a elemento.
<code>.^</code>	Potencia elemento a elemento.
<code>*</code>	Multiplicación matricial.
<code>\</code>	Resuelve sistemas lineales (<code>A\b</code>).



Use `.*`, `./`, `.^` para operaciones por elemento.

6 ATAJOS MUY UTILIZADOS

Expresión	Descripción
<code>A\b</code>	Resuelve $Ax = b$.
<code>A(:,j)</code>	Columna j de A .
<code>A(i,:)</code>	Fila i de A .
<code>A(:,end)</code>	Última columna de A .
<code>A(end,:)</code>	Última fila de A .
<code>A(i,j)</code>	Elemento en la posición (i, j) .
<code>v(end)</code>	Último elemento del vector v .



`end` hace referencia al último índice en cada dimensión.

7 RECUERDA

- ✓ Los índices comienzan en 1.
- ✓ Utilice `A\b` en lugar de `inv(A)*b`.
- ✓ Use `.*`, `./`, `.^` para operaciones elemento a elemento.
- ✓ Comente su código.
- ✓ Utilice nombres descriptivos para las variables.

8 FUNCIONES MÁS UTILIZADAS

<code>clc</code>	<code>clear</code>	<code>zeros</code>	<code>ones</code>	<code>eye</code>
<code>rand</code>	<code>size</code>	<code>length</code>	<code>sum</code>	<code>mean</code>
<code>diag</code>	<code>reshape</code>	<code>det</code>	<code>rank</code>	<code>norm</code>
<code>lu</code>	<code>qr</code>	<code>chol</code>	<code>eig</code>	



Curso CE1111 - Análisis Numérico para Ingeniería
Escuela de Ingeniería en Computadores



Profesor
Dr. Juan Pablo Soto Quirós



Correo
jusoto@tec.ac.cr



FORMULARIO DE COMANDOS

GNU Octave - Parte II

Funciones para cálculo numérico, simbólico y visualización.



GNU Octave

Lenguaje libre para el cómputo numérico y el análisis de problemas de ingeniería.

1



FUNCIONES ANÓNIMAS

Permiten crear funciones sin necesidad de guardar un archivo.

Sintaxis	Descripción
@(x)	Define una función anónima.
@(x,y)	Función anónima con varias variables.
@(x) x.^2 + 1	Ejemplo: función cuadrática.
@(x,y) sin(x) + y	Ejemplo: operaciones con variables.



Muy útiles como argumentos de otras funciones (fzero, quad, etc.).

5

COMANDOS MUY USADOS

Funciones generales que aparecen frecuentemente.

Función	Descripción
disp()	Muestra valores en la ventana de comandos.
fprintf()	Imprime texto formateado.
input()	Lee datos desde el teclado.
mod()	Resto de la división.
sqrt()	Raíz cuadrada.
sin()	Seno (radianes).
cos()	Coseno (radianes).
tan()	Tangente (radianes).
exp()	Exponencial e ^x .
log()	Logaritmo natural.



Estas funciones también aceptan vectores y matrices.

2



MÉTODOS NUMÉRICOS

Herramientas para resolver problemas numéricos de forma eficiente.

Función	Descripción
fzero	Encuentra una raíz de una función.
fsolve	Resuelve sistemas de ecuaciones no lineales.
fminsearch	Busca un mínimo local (Nelder-Mead).
fminbnd	Busca un mínimo en un intervalo [a, b].
quad	Integración numérica de una función.
quadgk	Integración mediante Gauss-Kronrod.
integral	Integración numérica moderna.
ode45	Resuelve EDO usando Runge-Kutta (4,5).
lsode	Resuelve sistemas de EDO.
polyfit	Ajuste de datos a un polinomio.
polyval	Evalúa un polinomio.
polyder	Derivada de un polinomio.
roots	Calcula las raíces de un polinomio.
fminunc	Minimización sin restricciones.



Revisar la ayuda de cada función para conocer argumentos y opciones adicionales.

3



CÁLCULO SIMBÓLICO

Permite manipular expresiones matemáticas exactas.

Función	Descripción
syms	Declara variables simbólicas.
sym	Convierte una expresión a simbólica.
diff	Calcula derivadas simbólicas.
simplify	Simplifica expresiones simbólicas.
expand	Expande productos y potencias.
factor	Factoriza expresiones.
subs	Sustituye valores en expresiones simbólicas.
solve	Resuelve ecuaciones simbólicas.
matlabFunction	Convierte una expresión simbólica en función numérica.
str2func	Convierte texto en función numérica.
double	Convierte simbólico a numérico.
fplot	Grafica funciones simbólicas.



Cargar el paquete con: `pkg load symbolic` si es necesario.

4



GRÁFICAS 2D

Funciones para visualizar datos y funciones.

Función	Descripción
figure	Abre una nueva ventana de figura.
plot	Grafica datos en 2D.
title	Agrega título al gráfico.
xlabel	Etiqueta del eje X.
ylabel	Etiqueta del eje Y.
legend	Muestra la leyenda.
grid on	Activa la cuadrícula.
hold on	Conserva la gráfica para añadir más.
hold off	Finaliza la superposición.
xlim	Define límites del eje X.
ylim	Define límites del eje Y.
clf	Limpia la figura actual.
axis equal	Iguala la escala de los ejes.
axis square	Hace cuadrada el área de la gráfica.
saveas	Guarda la figura en un archivo.
print	Exporta la figura (PDF, PNG, etc.).



Combine colores, estilos y marcadores para mejores visualizaciones.

7

ERRORES FRECUENTES

✗ Multiplicar matrices con * cuando se requiere elemento a elemento.	<code>A * B</code>	→	✓ Use <code>.*</code> para elemento a elemento.
✗ Dividir matrices con / cuando se requiere elemento a elemento.	<code>A / B</code>	→	✓ Use <code>./</code> para elemento a elemento.
✗ Potencias con ^ en matrices.	<code>A^2</code>	→	✓ Use <code>.^</code> para potencias elemento a elemento.
✗ Resolver $Ax = b$ con <code>inv(A)*b</code> .	<code>inv(A)*b</code>	→	✓ Use <code>A\b</code> (más estable).
✗ Olvidar que los índices comienzan en 1.	<code>A(0)</code>	→	✓ Los índices comienzan en 1.
✗ No verificar dimensiones en operaciones matriciales.	<code>A + B</code>	→	✓ Verifique tamaño de A y B.
✗ Sin punto antes del operador en operaciones elemento a elemento.	<code>A*B</code>	→	✓ Use <code>.*</code> , <code>./</code> o <code>.^</code> según corresponda.
✗ No cerrar figuras.	<code>figure</code>	→	✓ Use <code>close</code> o <code>close all</code> .



Curso CE1111 - Análisis Numérico para Ingeniería
Escuela de Ingeniería en Computadores



Profesor
Dr. Juan Pablo Soto Quirós



Correo
jusoto@tec.ac.cr

